

Lecture 25: Window Functions I

ROW_NUMBER, RANK, DENSE_RANK, NTILE

Dr. Innocent Nyalala
IIT Madras Zanzibar



D0.jpg

School of Engineering and Science
IIT Madras Zanzibar

5 May 2026

Lecture Outline

Today's Topics

- 1 Window functions vs. GROUP BY
- 2 OVER() clause anatomy
- 3 ROW_NUMBER()
- 4 RANK() vs. DENSE_RANK()
- 5 PARTITION BY — per-group ranking
- 6 NTILE(n) — quartiles & percentiles
- 7 Real-world: Instagram posts

Schema Used Today

```
ExamScores(  
    StudentID,  
    Name,  
    Subject,  
    Score,  
    Semester)
```

Window Functions vs. GROUP BY

GROUP BY — Collapses Rows

Name	Avg
Alice	78
Bob	82
Carol	91

10 students → 3 groups

Detail rows gone!

Window Function — Keeps ALL Rows

Name	Score	Rank
Alice	91	1
Bob	85	2
Carol	85	2
Dan	78	4
Eve	72	5

All rows kept + new column added!

Window function = compute something across rows without collapsing them

OVER() Clause Anatomy

RANK() OVER (PARTITION BY *col* ORDER BY *col*

Function

Window Keyword

- ▶ **PARTITION BY** — reset rank per group
- ▶ **ORDER BY** — defines ordering direction

- ▶ Both are **optional** in OVER()
- ▶ No PARTITION $\Rightarrow$ whole table = 1 window

ROW_NUMBER() — Unique Sequential Numbers

```
SELECT
  Name, Subject, Score,
  ROW_NUMBER() OVER (
    ORDER BY Score DESC
  ) AS RowNum
FROM ExamScores
WHERE Semester = 'S1';
```

- ▶ Always produces **unique** numbers
- ▶ Ties broken **arbitrarily**
- ▶ Great for **pagination**

Name	Subj	Score	Row#
Alice	Math	95	1
Alice	CS	95	2
Bob	Math	88	3
Carol	CS	82	4
Bob	CS	78	5

Tied 95s get different numbers!

RANK() vs. DENSE_RANK() — The Tie-Breaking Difference

Name	Score	RANK()	Gap?	DENSE_RANK()	Gap?
Alice	95	1		1	
Bob	95	1		1	
Carol	88	3	skips 2!	2	no skip
Dan	82	4		3	
Eve	78	5		4	

RANK() — Skips Numbers

2 people tied at rank 1

⇒ next rank is **3** (not 2)

Like Olympic medals: no silver if two golds

DENSE_RANK() — No Gaps

2 people tied at rank 1

⇒ next rank is **2**

Counts *distinct* score levels only

PARTITION BY — Rank Within Each Subject

```
SELECT Name, Subject,
       Score,
       RANK() OVER (
         PARTITION BY Subject
         ORDER BY Score DESC
       ) AS SubjectRank
FROM ExamScores;
```

	Math	Physics
restart	Alice 95 1	Bob 92 1
	Bob 88 2	Alice 87 2
	Carol 76 3	Dan 81 3

Each **Subject** is ranked **independently**

Bob ranks #2 in Math but #1 in Physics — same student, different windows!

NTILE(n) — Divide Into Equal Buckets

```
SELECT Name, Score,  
       NTILE(4) OVER (  
         ORDER BY Score DESC  
       ) AS Quartile  
FROM ExamScores;  
  
-- Q1 = top 25%, Q4 = bottom  
   25%
```

- ▶ Identify **top/bottom performers**
- ▶ Fair split — rows distributed evenly
- ▶ Use NTILE(10) for **deciles**

Q1 — Top 25% (score ≥ 90) 3 students

Q2 (80–89) 3 students

Q3 (70–79) 3 students

Q4 — Bottom 25% (< 70) 3 students

Real World: Rank Instagram Posts by Likes

```
-- Top post per user, per month
WITH RankedPosts AS (
  SELECT
    UserID, PostID, Month,
    Likes,
    RANK() OVER (
      PARTITION BY UserID, Month
      ORDER BY Likes DESC
    ) AS PostRank
  FROM InstagramPosts
)
SELECT UserID, PostID, Month,
       Likes
FROM RankedPosts
WHERE PostRank = 1;
```

📷 @alice_iitmz

Post A 1.2K #1

Post B 890 #2

Post C 450 #3

Partitioned by UserID + Month

- ▶ Same query finds **top post per user**
- ▶ PARTITION resets rank **per user per month**

Think-Pair-Share (5 min)

Problem

Using ExamScores(StudentID, Name, Subject, Score, Semester):

- 1 Find the **top 3 students per department** ranked by GPA (use RANK).
- 2 Show their **DENSE_RANK** as well — when do the two differ?
- 3 Use **NTILE(3)** to split students into low / mid / high tiers within each subject.

Starter Template

```
SELECT Name, Dept, GPA,
       RANK() OVER (
         PARTITION BY Dept
         ORDER BY GPA DESC
       ) AS R
FROM Students
WHERE R <= 3;  -- Can you do this?
```

Common Mistake

You **cannot** filter on a window function alias directly in WHERE.

Wrap in a CTE or subquery first!

Summary

Four Functions at a Glance

Function	Behaviour
ROW_NUMBER	Always unique
RANK	Gaps on ties
DENSE_RANK	No gaps on ties
NTILE(n)	Equal buckets

Key Clause

`OVER(PARTITION BY ...ORDER BY ...)`

- ▶ PARTITION = group boundary
- ▶ ORDER = sort within group
- ▶ Both optional

Next: L26 — Window Functions II (LAG, LEAD, Running Totals)